# SpanLink

## Design Document

Nicholas Alpert, Dylan Long,
Evan Marino, Ryan Toppin, Zachary Pizzuti

03/09/25
Updated: 4/30/25
CS 04400 - Spring 2025 - Section 2
GitHub Repository: https://github.com/Senior-Project-Rowan/SeniorProject.git

# Table of Contents

**Table of Contents**.....2
**High Level Description**.....3
**WIREFRAMES**.....4
Login Page:.....4
Signup Page (/signup):.....5
Dashboard Page (/dashboard):.....6
Patient Test Page (/patienttest):.....7
Survey Page (/survey):.....11
Aggregated Population Statistics Page:.....13
**MODELS & ADMIN PAGE**.....14
User:.....14
Test:.....14
Stimulus:.....15
Response:.....15
Plot:.....15
Survey:.....16
Notifications:.....16
Admin Page:.....16

# High Level Description

**Product Description:**

Our application aims to provide an efficient and effective web service to medical providers and patients for the administration of the Philadelphia Pointing Span Test or PPST. The PPST is a medical test that screens for possible neurocognitive impairments through a memory-based assessment involving letters and numbers.

**Provider Workflow:**

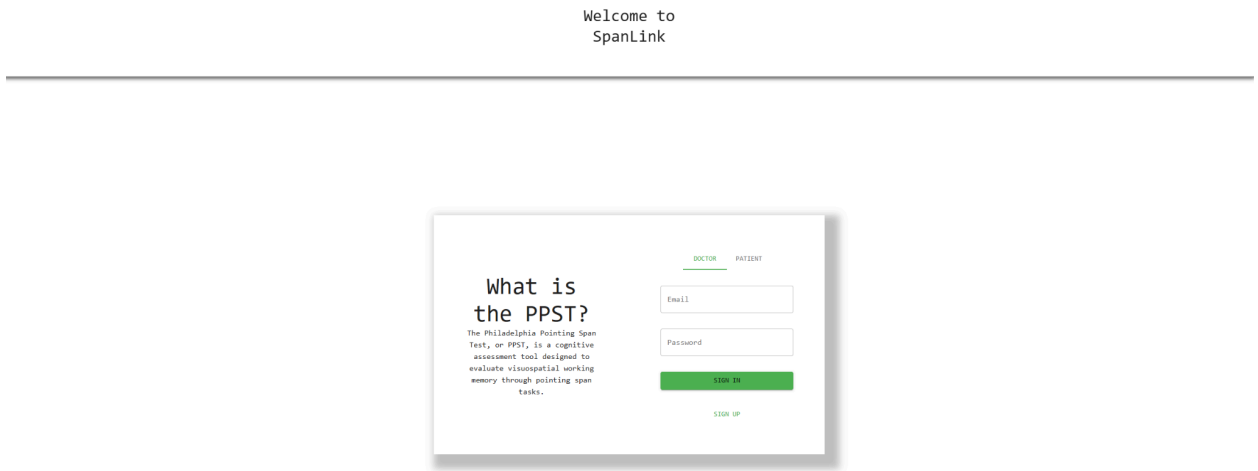
A provider/administrator of the test will be able to login or sign up from our homepage and then be routed to their dashboard. A link generator on the provider's dashboard will allow a provider to create a unique, one-time use link that generates a test for a patient. The provider can then send that link to a patient with the test in their preferred language with appropriate font size. In addition to creating links for patients, providers will also be able to view any ongoing tests or completed tests that they have created, export individual test data, and view aggregated statistics for all tests generated and individual metrics via an intuitive GUI and graph features.

**Patient Experience:**

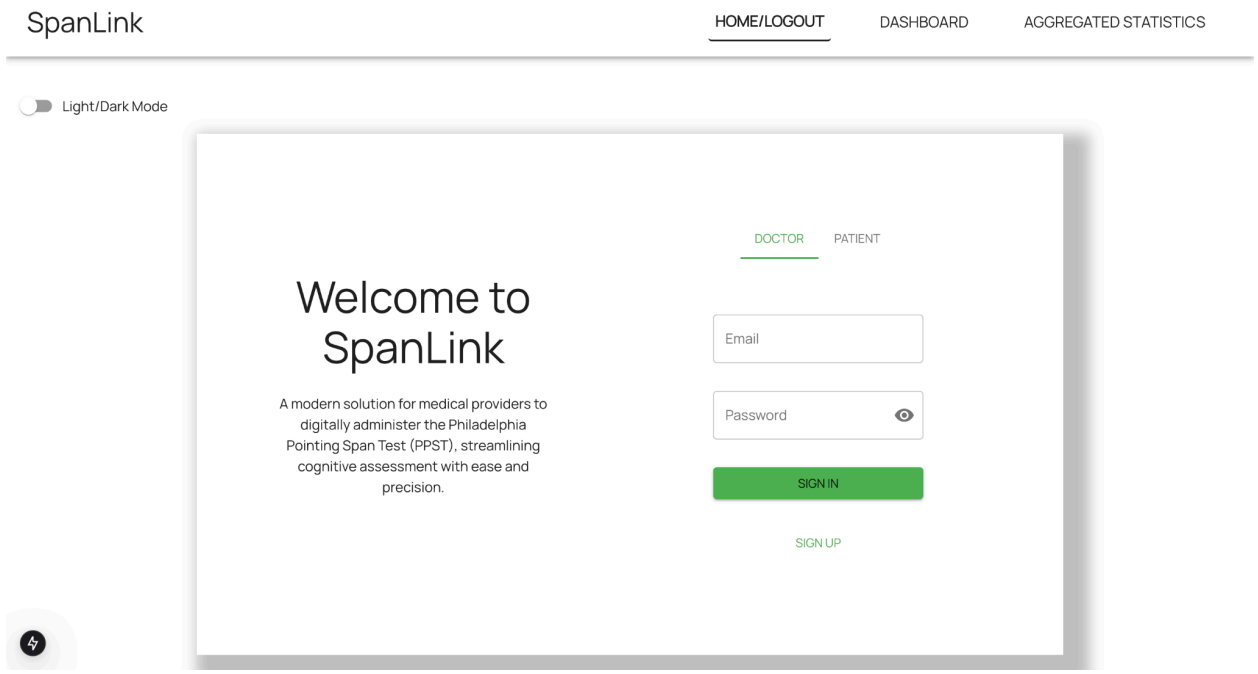
A patient will not be able to login or sign up to our application, as the patient will only be utilizing our application for diagnostic purposes. The administering provider will analyze and discuss results with the patient outside of our platform for now. When a user clicks their provided link, they will be directed to a web page that will deliver instructions as well as provide an optional practice test for the patient before they start the real test. On this page, patients will also be able to adjust any accessibility settings that would aid them with any disabilities, handicaps, or preferences that they may have, such as: color blindness, vision or hearing impairments, or language preference. Once the user has completed the test, they will be presented with the option to take a feedback survey to help us better serve our customers.

# WIREFRAMES

Original Login Page:




Updated Login Page:

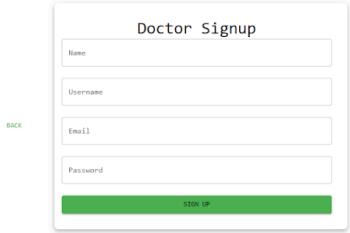



**Behavior Descriptions:**

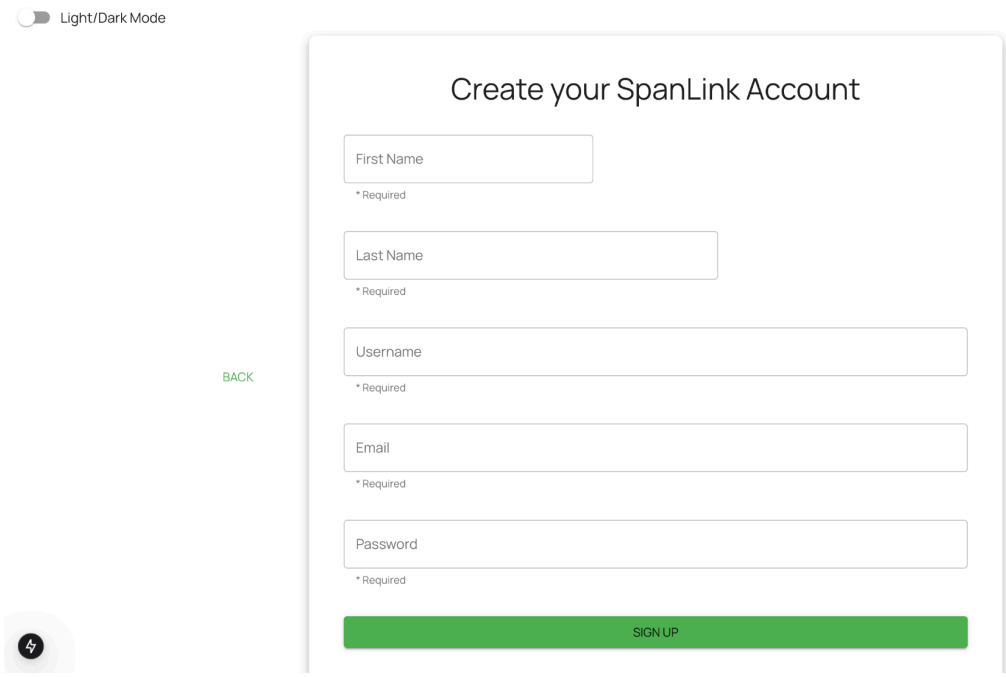
- The “DOCTOR” tab, selected by default upon loading the page, when clicked brings up the login prompt and widgets to sign in.

- Clicking the “PATIENT” tab will remove the sign in widgets and display a message for the user that instructs them to refer to their doctor for information.
- Clicking the “SIGN IN” button after entering valid doctor credentials will redirect the user to the /dashboard page.
- Clicking the “SIGN UP" button will redirect to the /signup page where the user can create an account.
- The Navigation Bar at the top is shown in the Login Page, Dashboard, and Aggregated Statistics Page, and clicking on any of them brings the user to the specified page, but only if signed in
- All pages also include a Light/Dark Mode switch that can be toggled, and is set to light mode as the default

**Signup Page (/signup):**
Original:



Updated:

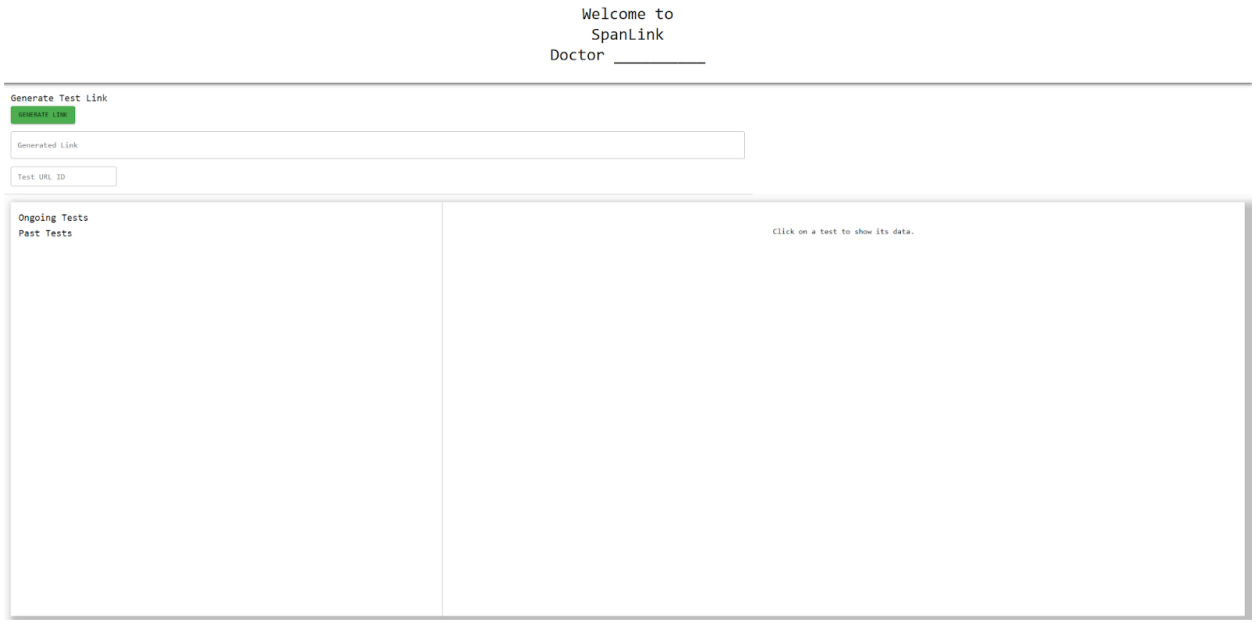



**Behavior Descriptions:**

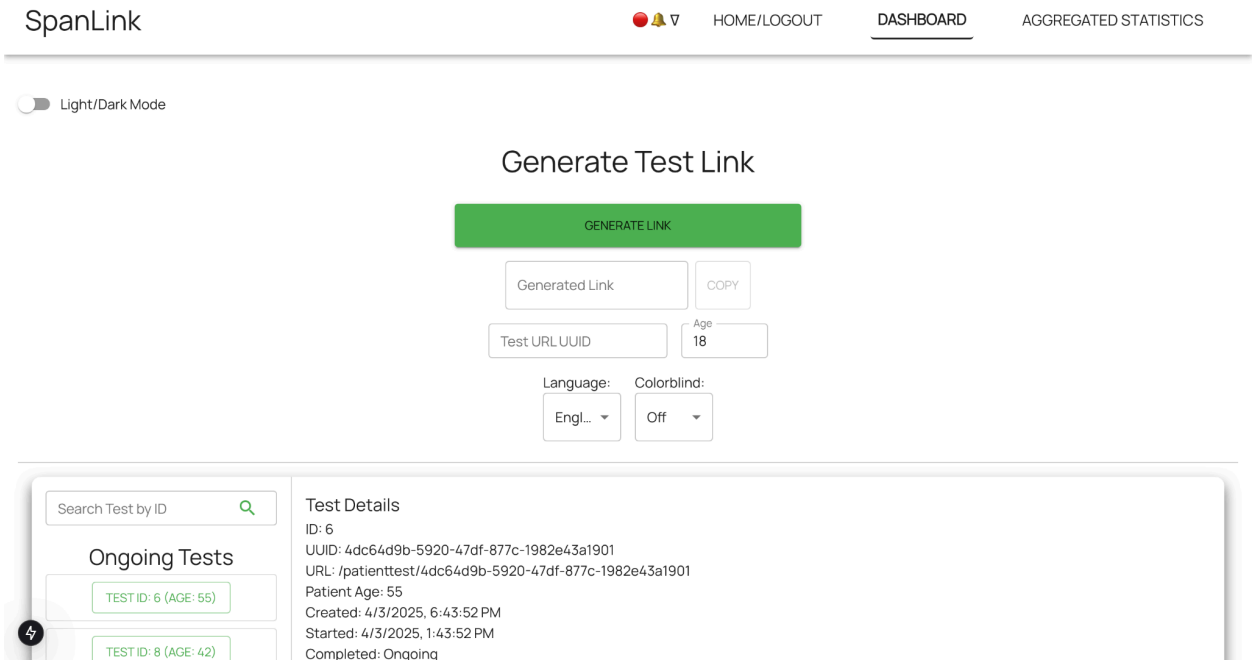
- Clicking the "SIGN UP" button will create an account for a user to access the dashboard with
- Doctors must fill in all fields to create an account
- A valid email address (includes an @ and .com/net/org etc.) is required

**Dashboard Page (/dashboard):**

Original:



Updated:



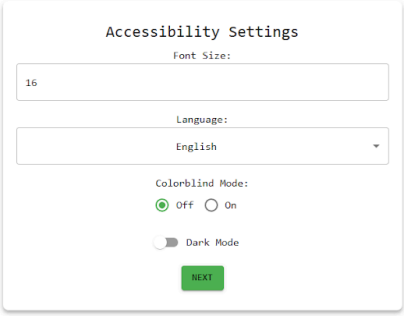

**Behavior Descriptions:**

- Clicking the "GENERATE LINK" button generates a sample test link with a sample test ID in each of their respective boxes
- Doctors must enter the patient's age before generating the link
- Doctors have the option to set the language and colorblind mode for the test before making the link, English and no colorblind mode is the default
- Doctors can click any of the Ongoing or Past tests to view individual statistics for those tests
- Doctors can search tests by their ID to view its statistics
- On the Navigation Bar, a bell icon can be clicked to give a notification of when a test has been completed

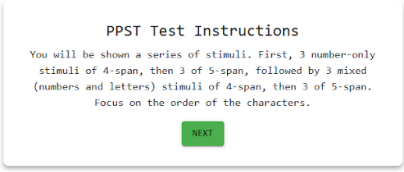
**Patient Test Page (/patienttest):**

Original:

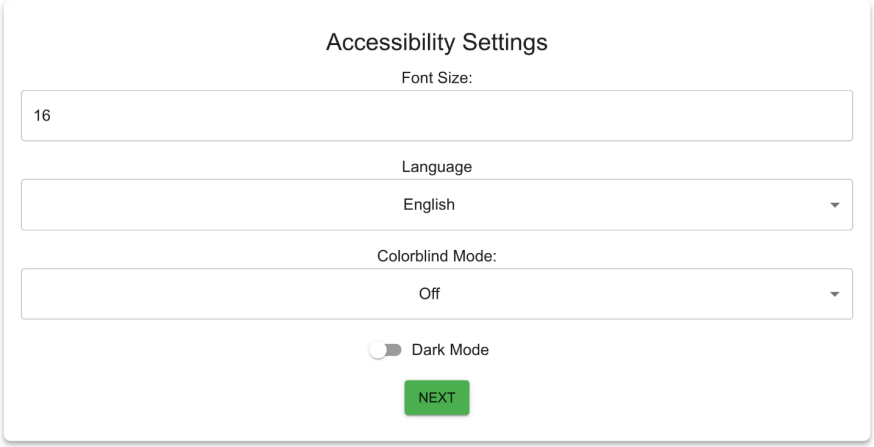
Welcome to
SpanLink

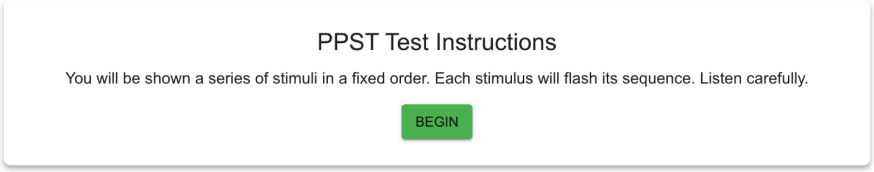



Welcome to
SpanLink




Updated:


Accessibility Settings
Font Size:
16
Language
English
Colorblind Mode:
Off
Dark Mode
NEXT


PPST Test Instructions
You will be shown a series of stimuli in a fixed order. Each stimulus will flash its sequence. Listen carefully.
BEGIN

Welcome to
SpanLink

Ready to Start

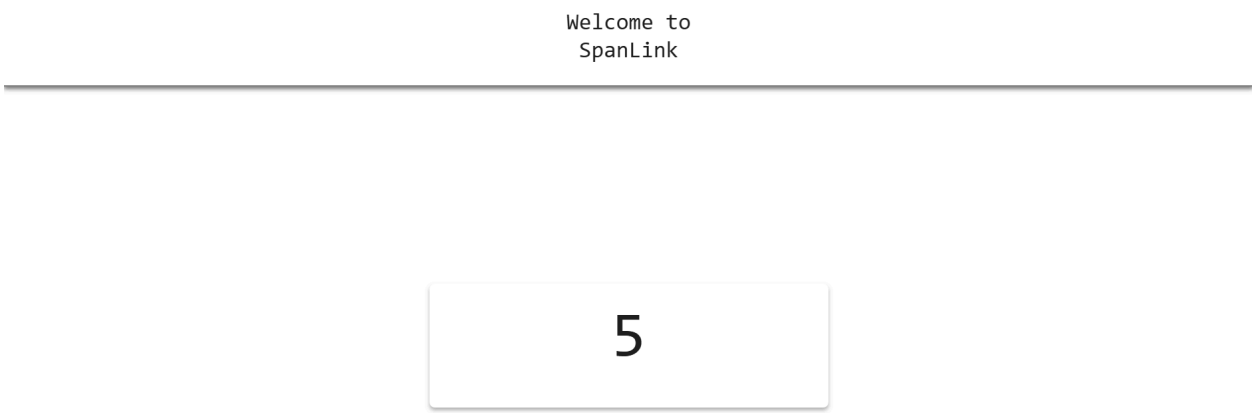
Press the start button when you are ready.

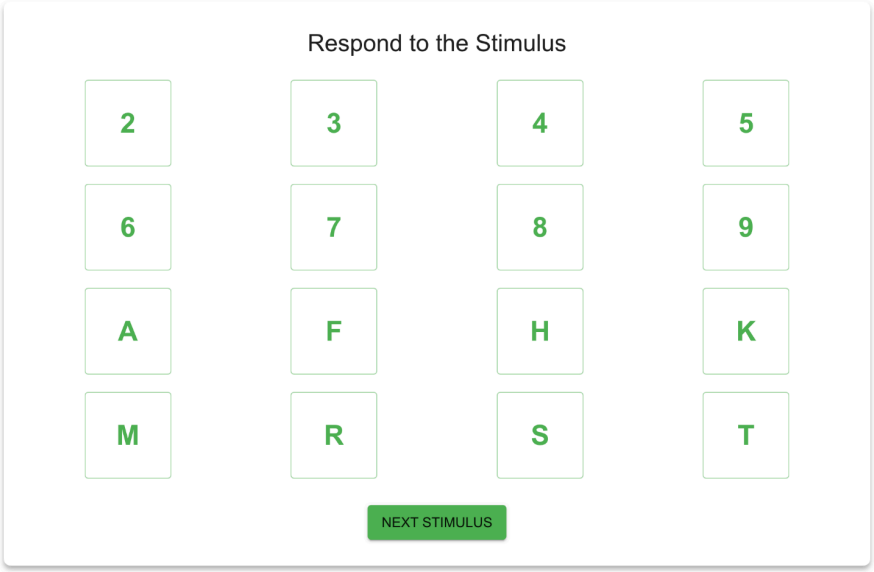
START TEST

Practice Stimulus

The stimulus you are about to respond to is a practice stimulus. Please answer carefully.

BEGIN

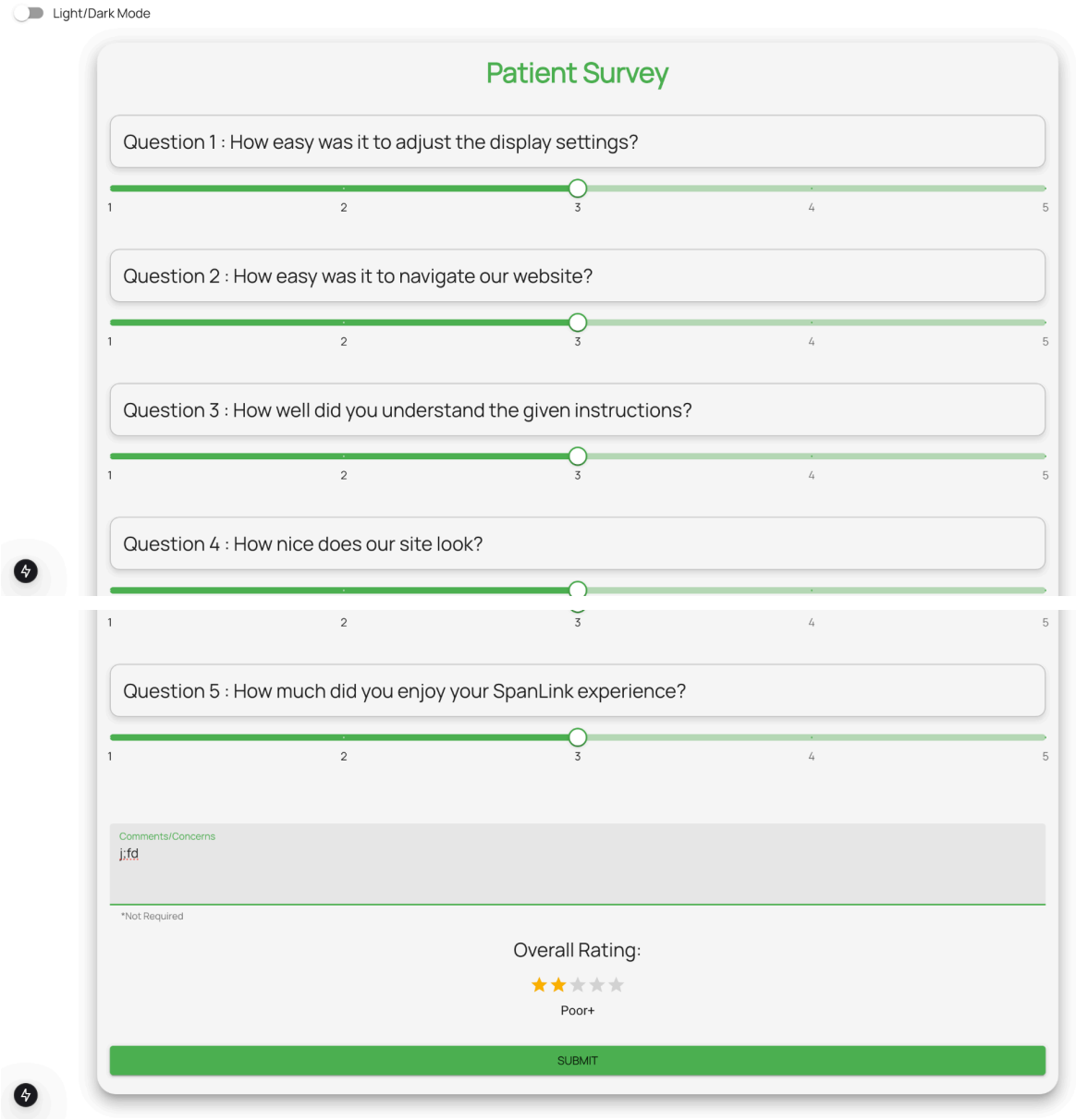


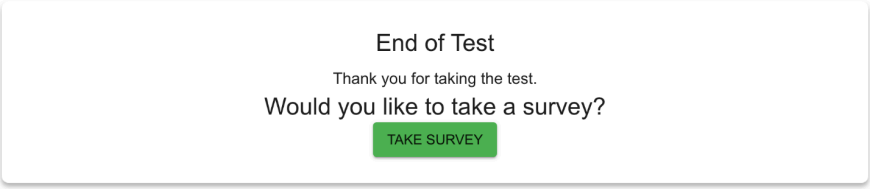




**Behavior Descriptions:**

- After the patient clicks their link, they are taken to an accessibility page, that lets them customize the font size, language, color blind options, and theme of the test
- They are then prompted to select “Next” and are given brief instructions on the PPST
- Selecting “Next” again will give them a “Start” button, which will begin the first Stimulus Flash
- The flash will let the patient if the test is a practice test
- The first Stimulus will then flash
- The patient will then be shown a “keyboard” of all available characters, where they can respond to the previous Stimulus Flash, as the page records their responses and latency

## Survey Page (/survey):






**Behavior Descriptions:**

- After the patient takes their test and the results are recorded, the user will be prompted to take an optional feedback survey. If they agree to take the survey, the user will press the “Take Survey” button on the /patienttest page, which will then route them to the /survey page.
- On the /survey page, there will be 5 questions regarding the patient’s experience, a textbox for optional comments or concerns, and a star rating scale of their overall experience.
- The 5 questions will be answered using a 1-5 rating scale on a slider, the textbox will show up to 5 lines of comment before it begins to scroll, and the star rating scale has increments of ½ for a more precise rating.
- When the user has completed the survey and pressed the “Submit” button, a card will be displayed saying “Thank you for your input, we appreciate your time and thoughtful feedback!”.

## Aggregated Population Statistics Page:

## Behavior Descriptions:

- Providers and patients can view global statistical visualizations and download the raw data in XLSX format.

# MODELS & ADMIN PAGE

**User:**

```
class User(AbstractUser):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    created_at = models.DateTimeField(auto_now_add=True)
    last_login = models.DateTimeField(null=True, blank=True)

    groups = models.ManyToManyField(
        "auth.Group",
        related_name="spanlink_users",
        blank=True,
    )
    user_permissions = models.ManyToManyField(
        "auth.Permission",
        related_name="spanlink_user_permissions",
        blank=True,
    )

    def __str__(self):
        return self.username
```

**Test:**

```
class Test(models.Model):
    assigning_provider = models.ForeignKey(User, on_delete=models.CASCADE,
related_name="tests")
    url = models.URLField(max_length=255)
    patient_age = models.PositiveIntegerField()
    def_language = models.CharField(max_length=10, blank=True, null=True)
    def_color_blind_mode = models.CharField(max_length=10, blank=True, null=True)
    stimuli_order = models.JSONField(blank=True, null=True)  # Using JSONField instead
of ArrayField
    created_at = models.DateTimeField(auto_now_add=True)
    started_at = models.DateTimeField(null=True, blank=True)
    completed_at = models.DateTimeField(null=True, blank=True)
    uuid = models.UUIDField(default=uuid.uuid4, unique=True, editable=False)


    def __str__(self):
        return f"Test {self.id} by {self.assigning_provider.username}"
```

**Stimulus:**

```
class Stimulus(models.Model):
    sequence = models.JSONField(blank=True, null=True)  # Using JSONField instead of
ArrayField
    correct_sequence = models.JSONField(blank=True, null=True)  # Using JSONField
    span_length = models.PositiveIntegerField()

    TYPE_CHOICES = [
        ('real', 'Real'),
        ('practice', 'Practice'),
    ]
    type = models.CharField(max_length=10, choices=TYPE_CHOICES)

    def __str__(self):
        return f"Stimulus {self.id}"
```

**Response:**

```
class Response(models.Model):
    test = models.ForeignKey(Test, on_delete=models.CASCADE, related_name="responses")
    stimulus = models.ForeignKey(Stimulus, on_delete=models.SET_NULL, null=True,
blank=True, related_name="responses")
    response_sequence = models.JSONField(blank=True, null=True)  # Using JSONField
instead of ArrayField
    sequence_latency = models.JSONField(blank=True, null=True)  # Using JSONField
    avg_latency = models.FloatField()
    is_correct = models.BooleanField()
    count_correct = models.PositiveIntegerField()

    def __str__(self):
        return f"Response {self.id} for Test {self.test.id}"
```

**Plot:**

```
class Plot(models.Model):
    title = models.CharField(max_length=255)
    image = models.ImageField(upload_to='plots/')
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

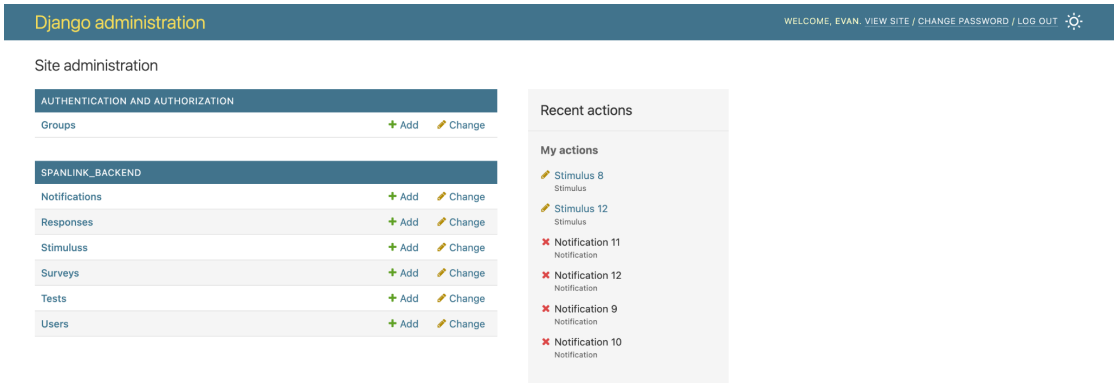
**Survey:**

```
class Survey(models.Model):
    # Survey ID = Autofield primary key
    survey_response = models.JSONField(blank=True, null=True)
    survey_rating = models.FloatField()
    survey_comment = models.TextField()

    def __str__(self):
        return f"Survey {self.id}"
```

**Notification:**

```
class Notification(models.Model):
    notification_id = models.CharField(max_length=255)

    def __str__(self):
        return f"Notification {self.id}"
```

**Admin Page:**